

Choosing the number of classes of a classification-bottleneck regressor by inference

Farhan Feroz
2026-06-14

Summary. This note concerns a regressor — a method that predicts a number y from an input x — that regularises itself by passing the prediction through a coarse description: instead of emitting one number it splits the output range into a set of classes and reports a probability-weighted blend of them. How many classes carry real weight at a given input — call it k — is a *resolution dial*: few classes is a coarse, heavily smoothed prediction, many is a fine, nearly unconstrained one. The dial should follow the data — few classes where the signal is weak and a fine split would only chase noise, more where it is strong enough to support detail, and a possibly different amount at each input. The model as built sets this dial with a hand-tuned penalty, and it does not work: k tracks whatever cap or penalty strength we pick, not the data (Section 2).

The central claim of this note is that the right amount of detail should not be chosen during fitting at all. It should be *measured afterwards*, by inference on data the model did not see. Fit a flexible mixture that is free to use many classes; then, at each input, ask a held-out question — does the full mixture predict fresh targets better than the single best bell-curve there? The size of that advantage, read locally around each input, is the per-input amount of genuine structure: the count by inference. It needs no knob, and it works on any flexible mixture, not a special one.

The note also develops, in full, the route it set out with: make the *fit* choose k , by treating the number of classes as unknown, putting a prior on the class weights that prefers few, and averaging the fit over that prior rather than fixing the weights at their best-fit values (Sections 3–6 and Appendix A). That route does work — on data whose conditional shape varies from input to input, a single weak prior on the dataset-wide average usage turns the dial where genuine structure appears (Sections 11–13). But two controlled experiments (Sections 14–15) show it is not what does the work. A plain flexible mixture with no such prior, read by the same held-out question, recovers the per-input count just as well, while standard point predictors and fixed-count mixtures cannot, because the count varies across inputs. Put on a like-for-like trial, the prefer-few prior earns nothing in the regime that matters and damages the reading where the classes are pinned in place. The recommendation — for the reader to weigh — is to drop it: keep the flexible mixture and the held-out readout, and let the prior go. None of this touches the model’s core — the classifier and the per-class heads; it changes only how their outputs are scored and removes the separate count-selector. It is still a regressor regularised through a classification bottleneck. What remains to test is the same method on real, many-dimensional data, where there is no known answer to check against (Section 16).

Notation

A single convention throughout: x is an input, y the scalar target to predict, and (x_i, y_i) for $i = 1, \dots, N$ are the N training pairs. The model has up to K candidate classes, indexed by $c = 1, \dots, K$ — $K - 1$ discretisation classes and, as the K -th, the bypass (Section 1 describes all three parts). The classifier turns each input into probabilities $\pi_1(x), \dots, \pi_{K-1}(x)$ over the discretisation classes — how strongly the input leans toward each. Class c then contributes a Gaussian (a bell-curve density) with mean μ_c and standard deviation σ_c from a small per-class head: for a discretisation class the head is fed only that class’s probability $\pi_c(x)$; for the bypass it is

fed the input directly. The Gaussian density of a value y with mean μ and standard deviation σ is written

$$\mathcal{N}(y; \mu, \sigma^2) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(y - \mu)^2}{2\sigma^2}\right), \quad (1)$$

and we abbreviate the value of class c 's density at training point i as $\phi_{ic} = \mathcal{N}(y_i; \mu_c(x_i), \sigma_c^2(x_i))$ — read it as “how well class c fits point i .” For a discretisation class, $\mu_c(x_i)$ and $\sigma_c(x_i)$ are shorthand for its head evaluated on the single number $\pi_c(x_i)$, so they reach the input only through the classifier; for the bypass they are its head's output on the input directly. The mixing weights (how much each class contributes to the blend) are written w_c ; these are a separate object from the classifier probabilities $\pi_c(x)$, as Section 4 makes precise. Throughout, capital K is the *cap* — the largest number of classes the model may use — and lowercase k is how many genuinely carry the prediction at a given input: not a setting we choose. Two ways of reading k appear in this note, and the difference between them is one of its lessons. The first counts how many class weights stay clear of zero — Sections 5 and 9 make this precise: the number of mean weights above a small fixed floor, with a continuous version (the effective number) reported alongside. The second, introduced in Section 11 and the one the note ends up trusting, asks on held-out data whether the extra classes actually predict fresh targets better than a single bell-curve. The two agree when the classes sit in fixed places, but the first over-counts when the classes are free to move — overlapping classes cost nothing in fit — so the held-out reading is the faithful one.

1. The model as built

The model has three parts, and it is worth stating each in plain terms because the rest of the note rests on what they do.

First, a **classifier**: a network that looks at the input x and outputs a set of non-negative numbers $\pi_1(x), \dots, \pi_{K-1}(x)$ that sum to one — how strongly that input leans toward each of the $K - 1$ discretisation classes (the bypass, below, is the K -th class and is reached differently). This is the classification bottleneck, and forcing the prediction through it is the regularisation the model is built around.

Second, a **per-class regression part**: each discretisation class c has a small head that is handed a single number — the classifier's probability $\pi_c(x)$ that the input belongs to class c — and from it produces that class's mean μ_c and width σ_c : where the class sits and how spread out it is. These heads see only their own class probability, never the raw input, so the only route by which the input reaches their means and widths is through the classifier (the bypass, below, is the exception — it reads the input directly).

Third, a **k-selector**: a separate network that outputs a distribution $p(k | x)$ over how many classes to use for this input — a split into k classes, for k from 2 (the coarsest split) up to the cap, or take the *bypass*: predict the target directly, stepping off the discretisation entirely. More classes is a finer, less-smoothed split; the bypass is the unconstrained option that passes through no split at all, so it is the most flexible, least-regularised choice. (Splitting into a single class would be no split at all — one class would hold every point — so the genuine splits start at two, and the bypass covers the no-discretisation case on its own.) The selector's job is to pick, for each input, how much smoothing the prediction should carry.

Two details of how these combine matter for the diagnosis. The splits are *nested*: using k classes means using the first k of the shared set. And the final prediction is assembled by a **weighted average of the per- k outputs**: each choice of k produces its own mean-and-width prediction, and these are blended using the selector's probabilities $p(k | x)$ into a single mean and a single width, which is then scored against the target. The number

of classes is steered by a separate term added to the training objective — either a fixed cost per extra class, or a penalty that pulls the selector’s distribution toward a chosen reference shape. Both carry a free strength that we set by hand.

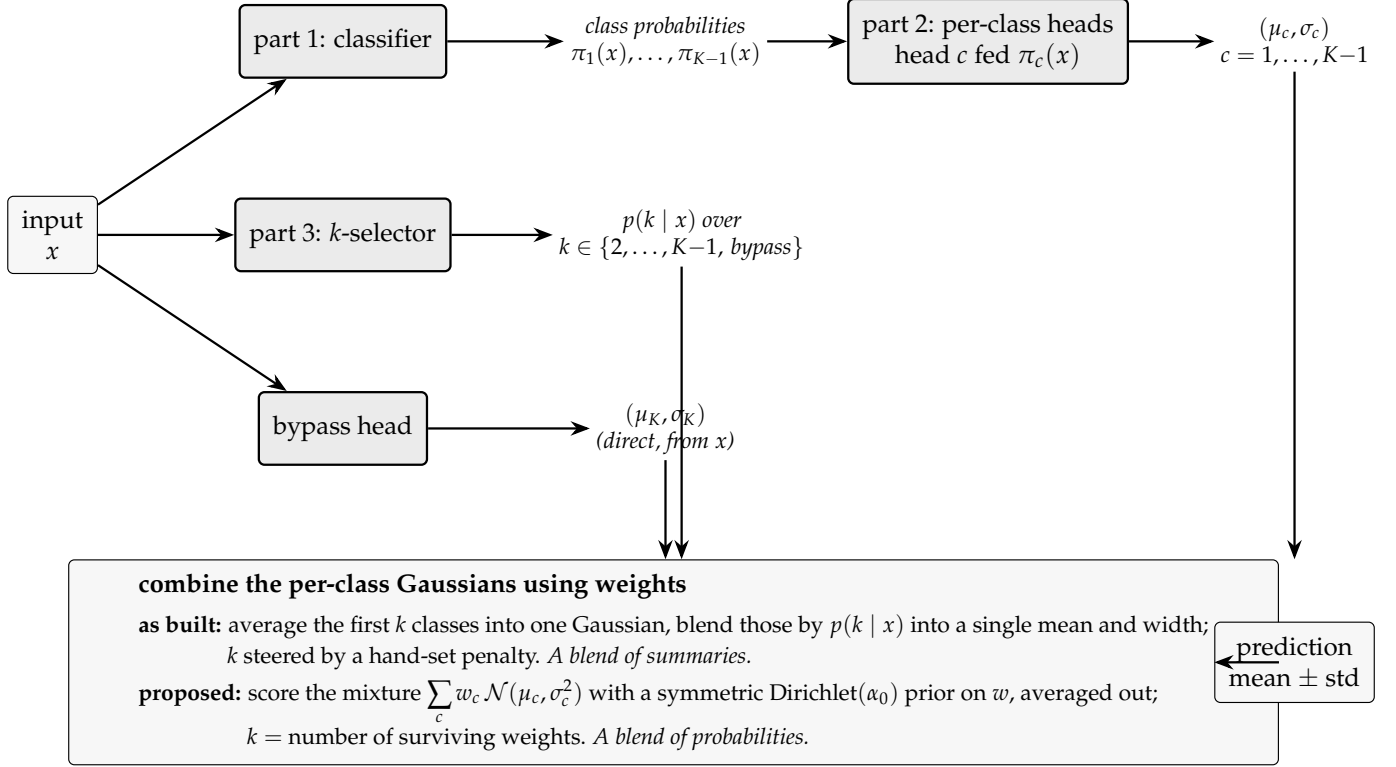


Figure 1: **The model and the two ways of combining its classes.** The input x feeds three parts: the classifier, which outputs the class probabilities $\pi_c(x)$; the per-class heads, each fed its own class’s probability $\pi_c(x)$ and producing that class’s mean and width (μ_c, σ_c) ; and the k -selector, which outputs a distribution $p(k | x)$ over how many classes to use, the bypass included. The bypass head predicts directly from x . The lower box contrasts the two readouts. As built (Section 2): average the first k classes into one Gaussian, blend those across k by $p(k | x)$, and steer k with a hand-set penalty — a blend of summaries. Proposed (Sections 3–5): score the genuine mixture and let a prior on the weights w , averaged out, decide how many classes survive — a blend of probabilities.

2. Why the choice of k does not work

Run the model as it stands and k does not track the data. Across a controlled study of it, with no steering term k settles near the midpoint of whatever cap is allowed — raise the cap and k rises with it, not with the data — and it does the same thing whether the data have real structure or are just a smooth trend plus noise. Its spread barely moves as the noise level changes. With a steering term it concentrates, but only where the hand-set strength puts it: tune the strength and k lands wherever you tuned it to. In short, the dial tracks the cap or the knob, never the signal.

The reason is structural, and it has a name in statistics: you cannot choose how complex a model should be by

how well it fits the data it was trained on. A finer split has more freedom, so it fits the training targets at least as well as a coarser one — never worse. “Fit the training data well,” taken alone, therefore always argues for more classes, never fewer. Left to the fit, k would climb to the finest setting; the only thing stopping it is the steering term, whose strength is arbitrary, so k simply tracks that strength.

There is a second, compounding problem specific to this model. The way the per- k predictions are combined — averaging their means and widths into a single bell-curve and scoring that — is not the probability the model assigns to the data under a genuine blend of classes. It is a blend of *summaries*. Two different blends of classes can have the same overall mean and width, so the averaged-summary score cannot tell them apart; the selector’s probabilities enter only as blending coefficients on those summaries, never as the weights of a genuine mixture whose probability is being evaluated. So the k -selector is not even connected to a quantity that could reward the right amount of detail — it is connected to a chosen reference shape through the steering term. There is, in the model as built, no quantity anywhere whose value falls *because k is finer than the data warrant*. Supplying exactly that quantity is the whole task.

3. The idea: score the full shape, then read the count from held-out data

The fix keeps the model’s one network — the classifier and the per-class heads — and changes two things: how the per-class pieces are combined, and how the number of classes is decided. The separate k -selector goes either way; what stands in for it is the subject of this section, and the answer the note reaches is the opposite of the one it starts with.

The change in combination is to score the model the way probability dictates: as the probability it assigns to each observed target under a genuine blend of its classes,

$$p(y_i | x_i) = \sum_c w_c \phi_{ic} \quad (\text{a mixture density}), \text{ not } \quad \mathcal{N}(y_i; m_i, v_i) \quad (\text{a blend of summaries}), \quad (2)$$

with $m_i = \sum_c w_c \mu_c(x_i)$ and $v_i = \sum_c w_c \sigma_c^2(x_i) + \sum_c w_c (\mu_c(x_i) - m_i)^2$ the mean and variance of that same mixture (the second is the law of total variance — the spread within classes plus the spread between their means). The left-hand form depends on the full shape of the blend, not just its mean and width, so it can tell a genuinely two-peaked target from a single broad one, because a finite mixture of Gaussians is pinned down by its full density, up to relabelling of its classes (which class is called which) (Teicher 1963); the right-hand form cannot, since two different blends can share one mean and one width — the degeneracy made concrete by the variance-matched twin of Section 11 and the curvature condition of Appendix B. Training on the left-hand form is what connects k to a quantity that actually depends on whether the detail is real.

How the number of classes is decided is the heart of this note. The instinct is to make the *fit* choose it: add something to the training objective that prefers few classes, so the model settles on the right number as it learns. We develop exactly that, in its principled form, because it is the right way to do it if it is to be done at all. Treat the number of classes as an unknown and put a **prior that prefers few** on the class weights; then — the one essential move — do not fix those weights at their single best-fit values, but keep a spread of plausible values consistent with the prior and the data and average the fit over that spread. Averaging is what charges the model for unused classes: a class the data do not need has no single setting that helps, so most of its plausible weight values contribute nothing and averaging drags the model’s probability down, while a class the data *do* need has its weight pinned high by the fit and pays no such cost. The prior carries one number — how strongly

it prefers few — a fixed, weak setting with a principled range (Section 5), not a strength tuned until the answer comes out at a target.

Why averaging and not a penalty term that rewards sparse weights? Because a penalty added to the best-fit weights is the same hand-set knob in new clothing, and worse: with a prior that strongly prefers few classes, the single most-likely weight setting is driven to empty classes by the prior itself, regardless of the data — the estimate collapses by construction. Keeping a spread of weights and averaging is precisely what avoids that collapse and lets the data, not the prior alone, decide. This is the same lesson that governs choosing the rank of a model elsewhere: a *per-input* most-likely weight setting cannot carry the complexity charge — it must be averaged over, or the prefer-few cost must be charged once on a quantity shared across the whole dataset rather than against each point alone, the route Section 13 arrives at.

But there is a simpler possibility, and it is the one the note ends up recommending. Suppose we do not try to make the fit choose the count at all. We let the mixture be flexible — free to use many classes — and fit it in the ordinary way. Then we *measure* the right number afterwards, on data the model did not see. At each input, compare the full mixture against the single best bell-curve there, on fresh targets, and take the difference in how well they predict: where the extra classes genuinely earn their keep the mixture predicts held-out targets better, and where they do not, it does not. That held-out advantage, read locally around each input, is the per-input count — recovered by inference, with no prior and no knob. Section 11 makes this precise and validates it against an exactly-computable answer; Sections 14 and 15 show it recovers the count as well as the prior route does — even on a plain flexible mixture with no prior at all — which is why the note recommends keeping the reading and dropping the prior.

Both routes need the genuine-mixture scoring above; they differ only in whether the count is chosen during the fit or read after it. The note develops the prior route first — Section 4 writes the model as a precise probability statement, Sections 5 and 6 derive the inference two ways (one shared set of weights for the whole dataset, and weights that vary per input), Section 7 compares them, Section 8 lists exactly what changes, and Section 9 fixes a test in advance — because that is where the machinery and the controlled tests were built. Sections 14 and 15 then put that route on trial against the plain reading.

4. The model written as a probability statement

The next ten sections develop the prior route in full: the model written as a probability statement (this section), the inference for shared and then per-input weights (Sections 5–6), what changes (Section 8), and a controlled test with its results (Sections 9–13). A reader who wants only the verdict can skip to Section 14, where this route is put on trial against the plain held-out reading and found unnecessary; what follows here is the route that trial tests.

We state the model as a recipe for generating the data, because the inference rules then follow from it with no extra choices. The recipe, for one shared set of weights (Section 6 relaxes this to per-input weights):

1. Draw the mixing weights $w = (w_1, \dots, w_K)$ from a prior that prefers few. The natural choice is the symmetric Dirichlet distribution — the standard prior over a list of non-negative numbers that sum to one — written

$$p(w) = \text{Dirichlet}(w; \alpha_0, \dots, \alpha_0) = \frac{\Gamma(K\alpha_0)}{\Gamma(\alpha_0)^K} \prod_{c=1}^K w_c^{\alpha_0-1}, \quad (3)$$

where Γ is the gamma function (the smooth extension of the factorial) and $\alpha_0 > 0$ is a single number, shared across classes, that controls the preference. When $\alpha_0 < 1$ the prior piles weight onto a few classes and pushes the rest toward zero — exactly “prefer few.” (The threshold is one here because each class is a scalar Gaussian; Section 5 gives the general rule, where the threshold sits, and why α_0 is a fixed weak setting rather than a tuned knob.)

2. For each training point i , draw a class label $z_i \in \{1, \dots, K\}$ with $\Pr(z_i = c) = w_c$.
3. Draw the target from that class: $y_i \sim \mathcal{N}(\mu_{z_i}(x_i), \sigma_{z_i}^2(x_i))$.

The means $\mu_c(x)$ and widths $\sigma_c(x)$ are produced by the heads of Section 1 — each ordinary class’s head from the classifier’s probability for that class, the bypass’s straight from the input. They are fitted, not given a prior: they are deterministic outputs of the network, with no free per-class location or width parameters to place a prior on, so the prior that does the pruning belongs on the weights instead. The weights w are the quantity we keep uncertain and average over, because they are what controls how many classes are used.

This is the bridge from the model as built. There, one network reported *which* class to use and a second reported *how many*, as a distribution over an integer. In this probability statement the count is not a separate integer to choose: the value of k is read straight off the weights — how many of them stay clear of zero — so “how many classes” becomes a consequence of which weights survive. The classifier the model already has is what produces these weights over all K classes; the separate k -selector is gone, its job absorbed into how many of those weights the prior leaves standing, with Sections 5 and 6 differing only in whether there is one weight list for the whole dataset or one per input. To be exact about the symbols: the classifier’s per-input numbers $\pi_c(x)$ of Section 1 are not the shared weights w_c . In the shared-weight case the per-input soft assignment is the *responsibility* r_{ic} , for which Section 5 gives a closed-form update from the fit and the current weights (one half of an alternation, not a one-shot solution). The classifier and the per-class heads are one network, trained together by gradient on the responsibility-weighted fit; the responsibilities and the global weights are read off in closed form from that network’s outputs, not from any separate network. In the per-input case the network instead produces the per-input weights directly, with no such closed-form replacement. Either way w is the mixing weight whose prior does the pruning.

If we wrote down only the single most probable w , multiplied by each point’s best-fit class, we would be back to a point estimate and the collapse described above. Instead the quantity we make as large as possible is the probability of the targets with the weights averaged out — the *evidence*,

$$p(y | x) = \int p(w) \prod_{i=1}^N \left(\sum_{c=1}^K w_c \phi_{ic} \right) dw. \quad (4)$$

This integral has no closed form, so we maximise a tractable lower bound on its logarithm instead — the standard route, derived from scratch in Appendix A. The bound introduces a stand-in distribution for the unknowns and is exact when that stand-in matches their true posterior. The classifier and heads the model already has are what carry this stand-in distribution — Sections 5 and 6 say how — and training them to maximise the bound is what makes k emerge from the surviving weights rather than from a penalty.

The bypass — predicting directly, with no split (Section 1) — is folded in as one further, always-available Gaussian class of the same form as the others, contributing its own $w_c \phi_{ic}$ term to the mixture; the only difference is that its mean and width are predicted straight from the input rather than tied to a discretisation region. We count it among the K classes — the discretisation splits draw on classes $1, \dots, K - 1$ (a split into k uses the first

k of them, $2 \leq k \leq K - 1$) and the bypass is the K -th — so every sum over c already includes it and nothing in the inference treats it specially. What the model as built treated as a separate selectable option is, here, simply a class always present in the mixture; “selecting the bypass” means the inference has put most of the weight on it. When it is the only class left carrying weight, the model is simply doing direct regression. And because the prior over the K weights is symmetric, the surviving classes need not be a prefix $1, \dots, k$ as the model as built required: the inference keeps whichever classes the data use — for a two-peaked target, the two classes sitting on the peaks, pruning the empty one between them — so k tracks where the structure is, not only how much of it there is.

5. Basis A: one shared set of weights

Here the weights w are a single list for the whole dataset. This is the clean case: the averaging has a closed form, and the k that survives is one number for the dataset.

Keep a stand-in distribution over the weights of the same Dirichlet shape as the prior, $q(w) = \text{Dirichlet}(\gamma_1, \dots, \gamma_K)$, with its concentrations γ_c to be learned, and a stand-in r_{ic} for the probability that point i came from class c (its *responsibility*, non-negative and summing to one over c). The lower bound on the log-evidence, derived in Appendix A, is

$$\mathcal{L} = \sum_{i=1}^N \sum_{c=1}^K r_{ic} \left(\ln \phi_{ic} + \mathbb{E}_q[\ln w_c] - \ln r_{ic} \right) - \text{KL}(\text{Dirichlet}(\gamma) \parallel \text{Dirichlet}(\alpha_0 \mathbf{1})), \quad (5)$$

where $\mathbb{E}_q[\ln w_c] = \psi(\gamma_c) - \psi(\sum_j \gamma_j)$ is the average of $\ln w_c$ under the stand-in (ψ is the digamma function, the derivative of $\ln \Gamma$; the identity is derived in Appendix A), $\mathbf{1}$ is the all-ones list, and the Kullback–Leibler divergence $\text{KL}(q \parallel p) = \mathbb{E}_q[\ln q - \ln p]$ is the standard non-negative measure of how far one distribution sits from another, zero only when they agree. Read plainly: a fit term (the responsibility-weighted data log-probability, through $\ln \phi_{ic}$), an assignment-spread term (the $-r_{ic} \ln r_{ic}$ summed over classes, an entropy that rewards spreading the assignment rather than forcing each point onto one class), and one cost — how far the learned weights have drifted from the “prefer few” prior. That single cost is the entire replacement for the hand-set steering term.

Maximising the bound gives two update rules that are applied in alternation; both are derived in Appendix A.

The responsibility update (holding the weights’ stand-in fixed):

$$r_{ic} = \frac{\phi_{ic} \exp(\psi(\gamma_c))}{\sum_j \phi_{ij} \exp(\psi(\gamma_j))}, \quad (6)$$

where the *effective* weight a class carries during assignment is $\exp(\psi(\gamma_c))$, not the plain posterior mean $\gamma_c / \sum_j \gamma_j$. The two differ in a way that matters: from the standard expansion $\psi(t) \approx \ln t - 1/(2t)$, the effective weight is $\exp(\psi(\gamma_c)) \approx \gamma_c \exp(-1/(2\gamma_c)) \approx \gamma_c - \frac{1}{2}$ for moderate γ_c , and collapses toward zero as $\gamma_c \rightarrow 0$, so a class with a small count is suppressed far harder than its raw share would suggest — that gap is what drives the pruning described below. (The plain posterior mean is still the right weight for reporting how much each class carries and for predicting on new inputs; the digamma form is what governs the assignment during inference.) Read it as: a point is assigned to a class in proportion to how well that class fits it, times how much overall weight the class currently carries. This responsibility needs the target y_i (through ϕ_{ic}), so it is a closed-form

quantity computed during training, not the output of the classifier, which sees only x — a distinction Section 7 returns to.

The weight update (holding the responsibilities fixed):

$$\gamma_c = \alpha_0 + N_c, \quad N_c = \sum_{i=1}^N r_{ic}, \quad (7)$$

where N_c is the soft count of points assigned to class c . The reading is clean: the prior contributes α_0 pseudo-counts to each class and the data contribute the real count N_c .

How a class prunes itself, and where the charge lives. Suppose some class is redundant — another already explains its points at least as well. Its soft count N_c is near zero, so its concentration γ_c sits near the prior floor α_0 , so its effective weight $\exp(\psi(\alpha_0))$ is tiny when $\alpha_0 < 1$ — for instance about 0.14 at $\alpha_0 = 0.5$ and about 3×10^{-5} at $\alpha_0 = 0.1$ — which shrinks its responsibilities further, which keeps its count near zero. The stable resting point is a switched-off class. The number that survive is k , and it is set by how much exclusive fit the data can supply, not by any hand-set strength. This is the charge from Section 3 made concrete: averaging the fit over the weights (the digamma effective weight, rather than the plain mean) is what docks a class the data do not use. This automatic emptying of unused classes is the established behaviour of variational Bayesian mixture model selection (Corduneanu and Bishop 2001); here it follows directly from the two update rules above rather than from any added term.

Why $\alpha_0 < 1$, honestly. The anchor is the standard analysis of fitting a mixture with more classes than the data need under a symmetric Dirichlet prior (Rousseau and Mengersen 2011): if α_0 is below half the number of free parameters per class, the surplus classes are emptied; above it, they are duplicated, which is unstable. A one-dimensional Gaussian class has two free parameters — a location and a width — so the threshold is $\alpha_0 < 1$, and a small value such as 0.1 or the choice $1/K$ sits safely inside the pruning regime. The honest caveat (Section 10): that analysis assumes the data are independent draws from one fixed mixture and that each class’s parameters are free numbers, whereas here each input has its own conditional classes produced by one shared network — so the exact threshold is a guide, not a guarantee, and the test in Section 9 confirms insensitivity to the value inside the regime.

What this basis buys and costs, and why it is only the grounding case. It buys clean, closed-form pruning with a single number to set weakly. But the mixing weights are one shared list, so two things follow. The surviving count is one number for the whole dataset, not a genuinely different k for each input. And — the more important point — the per-input mixing is global: the classifier here only shapes *where* each class sits (through the heads), while *how much* each class is weighted is the shared w , not a per-input quantity. Per-input weighting is exactly what the classification bottleneck is, so this basis does not exercise it. That makes Basis A the right place to confirm the pruning machinery works — on data where the mixing genuinely is global (Section 9) — but the per-input bottleneck doing the weighting, the heart of the prior route, lives in Basis B.

6. Basis B: weights that vary with the input

To let k genuinely differ from input to input — finer where this input’s region is clean, coarser where it is noisy — the weights must depend on x . This is the basis that puts the per-input classification bottleneck back to work — the classifier produces the per-input weights, the prior prunes them per input — at the cost of the closed form. It is the form the prior route is aiming at.

Now the classifier outputs, for each input, not a single weight list but the parameters of a *spread* of plausible weight lists — concretely the concentrations $\hat{\gamma}(x) = (\hat{\gamma}_1(x), \dots, \hat{\gamma}_K(x))$ of a per-input Dirichlet stand-in $q(w | x) = \text{Dirichlet}(\hat{\gamma}(x))$. The same averaging principle then applies per input: the contribution of input i to the objective is

$$\mathbb{E}_{q(w|x_i)} \left[\ln \sum_c w_c \phi_{ic} \right] - \text{KL}(q(w | x_i) \parallel \text{Dirichlet}(\alpha_0 \mathbf{1})), \quad (8)$$

a fit term averaged over the input’s own spread of weights, minus the same “prefer few” cost applied per input. Keeping the spread (rather than a single per-input weight list) is again what makes the charge real rather than a penalty in disguise: a per-input point estimate of the weights under $\alpha_0 < 1$ would collapse exactly as in the shared case.

The difficulty is that the first term — the average of the logarithm of a sum of weight-times-fit — has no closed form, because the average sits outside the logarithm. There are two standard ways through:

- **A genuine lower bound, via responsibilities.** Bring back a per-input soft assignment r_{ic} (a distribution over classes, exactly as in Basis A). For any such distribution the logarithm of a weighted sum is at least the weighted sum of the logarithms (Jensen’s inequality again), $\ln \sum_c w_c \phi_{ic} \geq \sum_c r_{ic} (\ln w_c + \ln \phi_{ic} - \ln r_{ic})$; averaging over $q(w | x_i)$ replaces $\ln w_c$ by $\mathbb{E}_q[\ln w_c]$, leaving the tractable contribution $\sum_c r_{ic} (\mathbb{E}_q[\ln w_c] + \ln \phi_{ic} - \ln r_{ic})$. This is a true lower bound — so the whole objective stays a lower bound on the evidence — tight when r_{ic} matches the input’s posterior assignment; the cost is the extra per-input responsibilities to carry.
- **A small average by sampling.** Draw a few weight lists from the per-input stand-in and average the log-fit over them directly, which is unbiased but noisier.

Either way there is no closed-form weight update like Basis A’s $\gamma_c = \alpha_0 + N_c$; the per-input concentrations are learned by gradient steps on this objective, because they are outputs of the network rather than free numbers with a matching-shape prior. The classifier is now exactly the network producing $\hat{\gamma}(x)$ — its per-input weights both mix the classes and, through their spread, carry the complexity charge — trained by the proper objective above rather than by a hand-set penalty. The separate k -selector is not rebuilt here; it is not needed.

What this basis buys and costs. It buys genuine per-input k and keeps the classifier as a full producer of input-dependent weights — the model the note set out to preserve. It costs the closed form: the weight inference is approximate and gradient-based. The “prefer few” cost is now charged once per input — one prior term against each point’s own fit — so no new strength is introduced beyond the same single concentration α_0 ; the honest catch is that against a single data point that prior term can weigh heavily, so α_0 does more work here than in the shared case. When this form is carried out, that is exactly what goes wrong — the weights collapse to one input-independent setting (Section 12) — and Section 13 repairs it by charging the prefer-few cost once for the whole dataset rather than once per input.

7. The two bases side by side

Question	Basis A (shared weights)	Basis B (per-input weights)
Does k vary per input?	No, one count for the dataset	Yes, a count per input
Weight inference	Closed form, exact	Approximate, gradient-based
Classifier role	Shapes class locations only; mixing is global	Produces the per-input mixing weights
Free number to set	One prior concentration α_0	One prior concentration α_0
Main risk	Too rigid if regions differ a lot	Looser bound or sampling noise

The two are rungs, not rivals — and the order matters. Basis A is the grounding rung: exact, one number to set, and it confirms the averaging-creates-a-charge machinery prunes at all, on data where the mixing genuinely is global. But it sidelines the per-input bottleneck, so it is a sanity check, not the destination. Basis B is where the per-input route lives: the only one of the two where the classifier keeps doing the per-input weighting. The prior route aims at Basis B, with Basis A run first only to ground the mechanism — though whether either is needed at all is what Sections 14–15 settle.

8. What changes and what stays

Stated plainly so the scope is unambiguous. **Stays:** the one network — the classifier and the per-class means and widths — and the whole idea of regularising through a classification bottleneck. The model itself is not changed. **Changes:** the per-class pieces are combined and scored as a genuine blend of probabilities rather than a blend of summaries; the separate k -selector is removed; and the number of classes is no longer steered by a hand-set penalty. What stands in for the penalty is the question this note settles: either a single weak “prefer few” prior folded into the fit (the route of Sections 4–13), or — the recommendation after the trial of Sections 14–15 — nothing in the fit at all, with the per-input count read off a held-out test instead. Either way no new tuned strength is introduced.

9. A controlled test designed before running it

The method is not to be trusted until it passes a test whose answer is known and whose pass/fail criteria are written before it runs. One point sets all the expectations: k measures the structure *beyond a single bell-curve* that the current noise level lets the data resolve. Smooth data with plain Gaussian noise has none of that — a single direct prediction (the bypass) already is the whole conditional — so the right answer there is one class at every noise level, not a sweep. A sweep in k appears only where the conditional is genuinely many-peaked, which one bell-curve cannot represent. The checks are built around that distinction: a positive sweep where structure is present, a negative control where it is not, and a shape test for the combination change.

Datasets. A *resolvable-modes* family: for each input the target is drawn from an equal-weight mixture of a known number of bell-curves, evenly spaced, with the spacing measured in noise-widths (the gap between neighbouring means divided by the noise standard deviation). That single number — how many noise-widths apart the modes sit — is the dial: small means the modes blur into one blob, large means they stand clearly apart. A *smooth* family: the target is a smooth function of the input (a single wave) plus Gaussian noise whose level we control — a single bell-curve conditional, nothing beyond its mean and width. And a *shape* pair built

to share the same mean and the same width but differ in shape: one target genuinely two-peaked, the other a single broad bell with the same mean and spread.

Check 1 (positive sweep) — does k follow how resolvable the structure is? On the resolvable-modes family, with the mixture held in one place (its location not varying with the input), sweep the spacing from well under one noise-width to several. Expectation, fixed in advance: the surviving number of classes rises from one (the modes merge into a single blob) to the true number (the modes stand apart). Pass if the surviving count is monotone in the spacing and moves from about one at the smallest spacing to about the true number at the largest; fail if it is flat, sits at the cap, or tracks the prior concentration α_0 . The bypass cannot short-circuit this: one bell-curve cannot stand in for several separated modes, so resolvable structure forces the model onto genuine classes.

Check 2 — is the prior a weak setting, not a knob? Re-run a resolved case (modes several noise-widths apart) at several values of the concentration inside the pruning regime (α_0 below one — for instance 0.1, 0.5, $1/K$). Pass if the surviving count is stable across these values (changes by at most one class); fail if it moves with α_0 the way it moved with the old hand-set strength, which would mean we had only renamed the knob.

Check 3 — does scoring the full shape matter? On the shape pair, compare the genuine blend-of-probabilities objective against the old blend-of-summaries one. Expectation: the blend of summaries cannot tell the two datasets apart (equal mean and width is all it sees), while the genuine blend recovers two classes on the two-peaked target and one on the broad one. Pass if the genuine objective separates the pair — two surviving classes versus one — and the summary objective does not; this is the direct demonstration that the combination change in Section 3 is load-bearing.

The negative control — does the method leave simple data alone? On the smooth family, with the bypass available, sweep the noise level. Expectation: the surviving count stays at about one — all weight on the bypass — at every noise level. The symmetric prior pulls every class toward zero equally, so it is the *fit* that breaks the tie: the bypass, reading the input directly, can match a smooth single-bell conditional that the discretisation classes — whose means barely move with the input — cannot, so its soft count stays high while theirs collapse to the prior floor and prune. One direct bell-curve already is the conditional, and the “prefer few” charge rewards using nothing more. Pass if k stays near one across the sweep; fail if it inflates the count on featureless data. This is the sharp contrast with the model as built (Section 2), which settled near the cap whether or not the data had structure: the new method should collapse here and engage classes only when there is structure to resolve.

Sequence — grounded first, then the real test. Run Checks 1–3 with the mixture held in one place to begin with. With the location fixed, the input carries no information and the data are independent draws from one fixed mixture — exactly the setting the pruning analysis of Section 5 assumes, and whose assumption Section 10 flags. So this is the right place to confirm the mechanism and the implementation: a *necessary* check, but not a sufficient one, because in this regime the method reduces to a standard variational mixture whose behaviour is already understood. The genuinely new question is the next rung — let the mixture’s location move with the input, so the shared network must produce the classes from the class probabilities, the regime the borrowed threshold does not cover and where success is not a foregone conclusion. Letting the *number* of modes itself vary by region (per-input k , Basis B) is the last rung.

What to record. For each run: the surviving number of classes — those whose mean weight $\bar{w}_c$ (the plain posterior mean of Section 5, $\bar{w}_c = \gamma_c / \sum_j \gamma_j$ for one shared set) exceeds a small fixed floor — we use $1/(2K)$, half the weight a class would carry if all K shared equally. The pass/fail criteria above are read off this integer

count. Report alongside it, as a continuous cross-check, the effective number $\exp(H)$, where $H = -\sum_c \bar{w}_c \ln \bar{w}_c$ is the entropy (natural logarithm) of the mean weights — one when all weight sits on a single class, K when spread evenly across all of them. Also record the held-out probability the model assigns to fresh data under the genuine mixture, and the recovered shape on the shape pair. Keep these so the checks can be re-read without re-running.

10. What this does not yet prove

Three honest limits, so the review knows what the test is for.

The pruning argument of Section 5 shows that a switched-off class is a *stable resting point* of the update rules; it does not prove that training reaches the right number from any starting point, nor that it is immune to the shared network keeping a class alive by reusing it elsewhere. The means and widths are fitted by gradient while the weights update, so the clean alternation of the textbook analysis is only approximate here.

The $\alpha_0 < 1$ threshold is borrowed from a setting with free per-class parameters and independent draws from one fixed mixture; with a shared network producing input-dependent classes, it is a guide, and Check 2 is what turns “insensitive to the exact value” from a hope into evidence.

That the surviving k tracks the signal-to-noise ratio is the central promise, and it is a prediction, not yet a theorem — Check 1 is built to falsify it. The plan is sound on the mathematics of the bound and the update rules (Appendix A); whether it behaves as intended under a jointly-trained shared network is exactly what the controlled test is for, which is why the test comes before the trust.

Sections 11–13 report those tests run for the prior route, and Sections 14–15 then put the prior itself on trial against the plain held-out readout. The signal-to-noise tracking the central promise predicts holds on controlled data. What stays open is what was open before — a guarantee that training reaches the right number from any starting point — together with the question these later results answer in the negative: whether the “prefer few” prior is needed at all. The results throughout are on small controlled data with a one-dimensional input, where the true answer is known; the same method on real, many-dimensional targets, where it is not, is the open next step (Section 16).

11. Carrying it out: data whose shape varies with the input

Section 9 ran its checks with the mixture held in one place — the grounding regime, where the data are independent draws from one fixed mixture. The genuinely new question it named, the last rung, is to let the conditional shape vary from input to input and ask whether a separate amount of detail is chosen for each. That needs data built for it and a check that can read a separate answer at each input.

The data. For each input x in the unit interval the target is drawn from an equal-weight pair of bell-curves of common width σ — here σ is the width of the noise in the data, a fixed property of the construction, not a fitted quantity — their centres a distance $s(x)\sigma$ apart, with the spacing growing steadily with the input,

$$s(x) = s_{\min} + (s_{\max} - s_{\min})x, \quad s_{\min} = 0.3, \quad s_{\max} = 4, \quad (9)$$

from 0.3 of a noise-width at one end to 4 at the other. Their shared centre is held fixed, so the *only* thing that

changes across inputs is the shape: at small x the two bell-curves overlap into a single blob, at large x they stand clearly apart. Two equal-weight bell-curves of equal width σ whose centres are a distance d apart form a genuinely two-peaked density exactly when $d > 2\sigma$ — Appendix B derives this from the sign of the density’s curvature at the midpoint. So the spacing crosses from one effective peak to two at the input x^* where $s(x^*) = 2$, a boundary fixed by the construction and known before any model is fitted; here $x^* \approx 0.46$.

The trap. Alongside it, a single-peak twin built to be a fair decoy: for each input a single bell-curve with the *same mean and the same variance* as the two-peak target at that input. The two-peak target’s variance is $\sigma^2 (1 + s(x)^2/4)$ — the within-class spread σ^2 plus the spread between the two centres, $(d/2)^2 = \sigma^2 s(x)^2/4$ — so as the spacing grows the target widens, and the twin widens in exact step but never grows a second peak. Anything that merely tracks the overall spread treats the two as identical; only something reading the full shape can separate them. This is the per-input form of the shape pair of Section 9.

Reading a separate answer at each input. The honest measure of whether the extra detail is worth it is the probability the model assigns to data it was not trained on (Section 9, what to record). Per input this hits a wall: each input carries a single held-out target, far too noisy to score on its own. The way through is to compare, at each held-out point i , the model’s mixture against a single bell-curve predictor trained separately on the same data, and record the difference of their negative log-probabilities,

$$\Delta_i = \ell_i^{\text{one}} - \ell_i^{\text{mix}}, \quad \ell_i^{\text{mix}} = -\ln \sum_c w_c(x_i) \phi_{ic}, \quad (10)$$

with ℓ_i^{one} the single bell-curve’s negative log-probability at the same point; Δ_i is positive where the mixture predicts the held-out target better than one bell-curve can. One Δ_i is noisy but unbiased: its average over points near a given input estimates the true advantage there, and averaging neighbours cancels the noise without biasing the estimate, because the input-dependence sits in the two models, not in the averaging. The curve $\hat{\Delta}(x)$ is that local average — here formed by averaging the per-point differences within narrow equal-width bins of the input.

Because this is controlled data, the answer the curve estimates can also be computed exactly: draw many targets at a fixed input and average the same difference, giving a curve $\Delta^*(x)$ with no sampling noise and no neighbour-averaging. Confirming that $\hat{\Delta}(x)$ matches $\Delta^*(x)$ — they agree to about a hundredth of a nat, the nat being the unit of log-probability when natural logarithms are used — is what licenses the cheap one-target estimate for the real setting, where only one target per input is ever available.

12. The first form of per-input weights collapses, and why it is not a missing constant

The per-input construction of Section 6 — give each input its own spread of weights with the prefer-few cost charged per input — was carried out on this data, with the bell-curve centres held fixed so the only way to move probability between classes as the input changes is through the weights. It collapses: the fitted weights come out essentially the same for every input, and even the same across the two-peak data and its single-peak twin. The per-input dial does not turn.

The cause is the one Section 6 flagged. The per-input objective pairs, for each input, one point’s worth of fit against a full prefer-few cost. Measured on this data the cost outweighs the fit by roughly three to one, so the cost — which pulls every input’s weights toward the same prior shape regardless of the data — dominates, and the weights settle at a single input-independent setting.

It is worth being exact that this is **not a missing normalising constant**. The per-input objective is the correctly-scaled lower bound for the model it describes — the model in which each input draws *its own* weights, so one point of fit is paired with one prior cost, and that pairing is right for that model. The imbalance is a modelling consequence, not an arithmetic slip. Basis A escaped it because there one prior cost was shared across all N points: the share borne by each point is the prior cost divided by N , negligible against that point’s fit. Giving every input its own prior multiplies that share back up by N . Rescaling the per-input cost by $1/N$ by hand would cancel the factor, but a bare $1/N$ is an unmotivated strength — the very thing the note set out to avoid. The principled move is to put a single prior on the one object that is genuinely shared across the whole dataset, which is the next section.

13. The prefer-few prior turns the dial — and points past itself

Keep the per-input weights, but charge the prefer-few cost only once. The weights $w(x) = (w_1(x), \dots, w_K(x))$ are now read straight off the classifier as a plain per-input setting — non-negative and summing to one — with no per-input spread to average over; the data are scored by the genuine per-input mixture $\sum_c w_c(x_i) \phi_{ic}$. The single place the prefer-few preference enters is a prior on the **average usage** across the dataset,

$$\bar{w}_c = \frac{1}{N} \sum_{i=1}^N w_c(x_i), \quad (11)$$

the mean weight class c carries over all inputs. A symmetric Dirichlet prior with the same single concentration $\alpha_0 < 1$ is placed on $\bar{w} = (\bar{w}_1, \dots, \bar{w}_K)$, and the objective is the data’s log-probability plus that one prior term — the logarithm of the likelihood times the prior, whose most-probable setting we seek (a maximum-a-posteriori estimate), so it needs no derivation beyond writing the two factors down,

$$\sum_{i=1}^N \ln \sum_c w_c(x_i) \phi_{ic} + \ln \text{Dirichlet}(\bar{w}; \alpha_0 \mathbf{1}), \quad (12)$$

whose usage-dependent part is $(\alpha_0 - 1) \sum_c \ln \bar{w}_c$ — with $\alpha_0 < 1$ this rewards piling the *average* usage onto a few classes and emptying the rest, exactly prefer-few, now a statement about the dataset as a whole rather than about any single input.

This restores the balance Basis A had and the per-input form lost: N points of fit against one prior cost, the per-point share of that cost again $1/N$ and weak — but with the weights free to differ from input to input. It is a single best setting of the per-input weights rather than a spread averaged over, and the collapse of Section 12 does not recur, not because of any averaging but because the prior now sits on the shared average usage, so no single point is asked to carry it alone.

This may look like it contradicts Section 3, which argued that a single best setting of the weights under a prefer-few prior collapses, and that one must average over a spread instead. The difference is in what decides how many classes are used. Section 3’s collapse is what happens when the *prior* is doing the deciding: choose the single weight setting that maximises its own prior probability under a prefer-few prior, and the prior alone drives unused classes to zero, data or no data. Here the prior is demoted to a weak, dataset-wide nudge — charged once across all N points, so the pull it exerts on any one input’s weights is that cost divided by N , far too small to override the fit — and the deciding is done by the fit, which sets each input’s weights from that

input’s own data. Averaging the weights out, as Basis A does, and charging the prior weakly across the whole dataset, as here, are two routes to the same safeguard: both keep the prior from dominating a single best setting. Basis A took the first route because its weights were one shared list, with nothing to spread the prior against; per-input weights supply that spread directly, so the second route is open.

Run on the data of Section 11, the repaired model turns the dial.

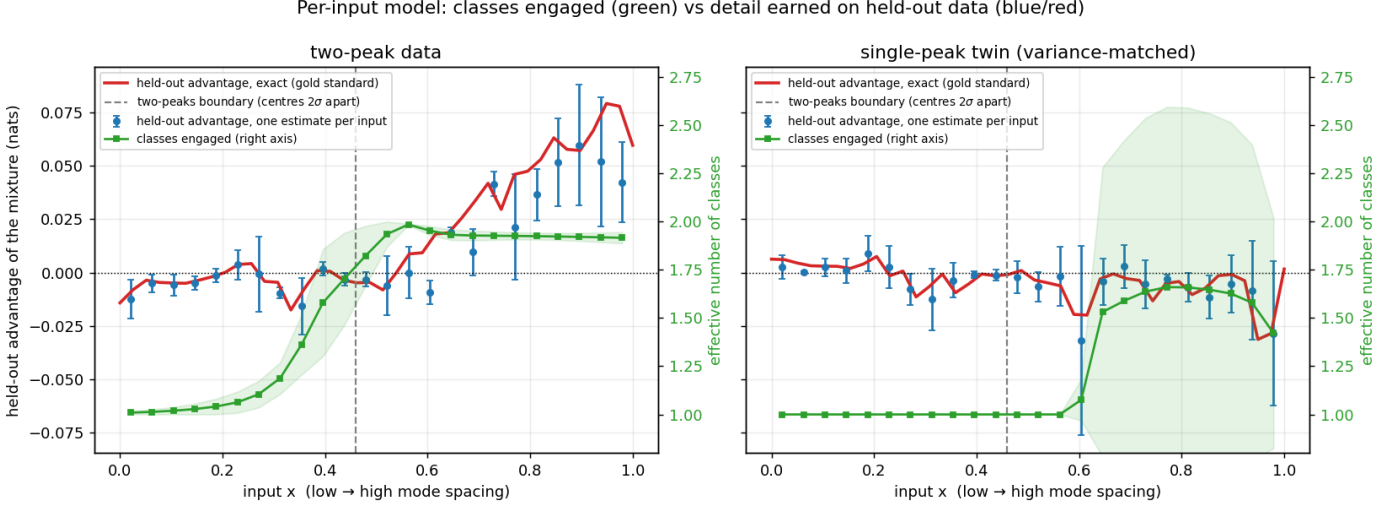


Figure 2: The repaired per-input model: detail engaged versus detail earned. Left, the two-peak data; right, the single-peak twin, whose spread grows identically with the input. Green, right axis: the effective number of classes the model engages at each input — one when all the weight sits on a single class, larger when it is spread across several (the entropy-based measure of Section 9, here formed from the per-input weights $w(x)$ rather than the dataset mean). Blue points and red line, left axis: the held-out advantage of the mixture over a single bell-curve, in nats — the cheap one-target estimate $\hat{\Delta}(x)$ (points) and the exact gold standard $\Delta^*(x)$ (line). The dashed line marks the known two-peaks boundary x^* , where the centres reach 2σ . On the two-peak data the count rises from about one to about two across the boundary and the held-out advantage turns positive there; on the twin the count creeps up at the widest spacings but the held-out advantage stays flat at zero — over-claimed detail the held-out check refuses to credit. Shaded bands show the spread across repeated runs from different random starts; error bars on the points are the spread of the one-estimate curve.

On the two-peak data the effective number of classes engaged rises from about one where the bell-curves overlap to about two where they stand apart (here from 1.0 to 1.9), with the rise centred on the known boundary x^* ; and the held-out advantage of the mixture, near zero below the boundary, turns positive above it, averaging about $+0.035$ nats over the two-peak region and reaching about $+0.07$ nats at the widest spacing — the engaged classes earn their keep exactly where genuine structure appears. On the single-peak twin, whose spread grows identically, the held-out advantage stays within a hundredth of a nat of zero throughout: the extra detail is never worth anything, because there is none. The count there is not perfectly held at one — beyond the midpoint it drifts up to around one and a half, with wide run-to-run spread (the shaded band), as the model is tempted by the growing spread — but the held-out check withholds all credit, which is the point: the count alone can mislead, and the held-out advantage is the honest arbiter. Across both panels the cheap one-target curve tracks the exact gold-standard curve, as the validation of Section 11 requires.

This is the per-input bottleneck of Section 6 doing its job: a different amount of detail at each input, set by the data, with the held-out check confirming the detail is real where the model engages it and absent where it is not. But notice which of the two readings did the honest work. The engaged-class count crept up on the single-peak twin where there was nothing to find; it was the held-out advantage that refused the credit. That last point — that the engaged count can mislead while the held-out advantage stays trustworthy — is the thread the rest of the note pulls. If the held-out advantage is the reading to trust, perhaps it, and not the particular prior that produced this fit, is the contribution. Sections 14 and 15 test exactly that, and the limits of Section 10 — small controlled data, one-dimensional input — still stand over everything that follows.

14. A like-for-like comparison against standard methods

Three families of controlled data make the per-input count known in advance, so any method's count can be graded against the truth. In all three the target at input x is drawn from an equal-weight set of bell-curves of common width σ (the noise width), and the construction fixes how many are genuinely resolvable at each input:

- a *rising-spacing* family (the data of Section 11): two bell-curves whose separation grows with the input, so the resolvable count goes from one (overlapped) to two (well apart);
- a *staircase* family: the number of bell-curves itself steps up with the input — one, then two, then three — so the true count is 1, 2, 3 across the range;
- a *hump* family: two bell-curves through the middle of the range and one at each end, so the true count rises from one to two and falls back to one. This is the decisive shape, because no reading that merely tracks the input coordinate could reproduce a count that goes up and then down; only reading the structure can.

On each family we compare three predictors, on held-out data, against the best achievable — the probability the true generating process itself assigns, which no method can beat:

- an everyday point-prediction method (gradient-boosted regression trees) reporting a single value and one uncertainty band;
- a standard mixture model trained at a *fixed* number of components, with that number chosen by held-out tuning — a mixture density network, a network whose outputs are the weights, means and widths of a fixed-size mixture;
- the flexible mixture of this note, read by the held-out question of Section 11.

Overall fit is the average negative log-probability on held-out data (lower is better). The first column below is an absolute level — the lowest average negative log-probability any method could reach, set by the true generating process — and the next three columns are *gaps* above that level, so zero would be unbeatable and smaller is better. The last column is the single count the fixed-count mixture's held-out tuning settles on, on each of three random starts.

Family	Best achievable (level)	Point method (gap)	Best fixed mixture (gap)	This note's mixture (gap)	Fixed-mixture count, three starts
Rising spacing	+0.569	+0.235	+0.009	+0.024	2, 2, 2
Staircase	+0.771	+0.406	+0.023	+0.027	6, 3, 4

Hump	+0.566	+0.236	+0.011	+0.022	2, 2, 2
------	--------	--------	--------	--------	---------

The point method sits far from the best achievable everywhere — a gap of +0.24 to +0.41 nats — because on data with several bell-curves its single value lands between them, where little data actually falls. A fixed-count mixture, given enough components, matches the best achievable on overall fit (a gap of at most +0.02 nats). But it must commit to one count for the whole dataset, and on the staircase family, where the true count varies across inputs, held-out tuning cannot find a single right number: it picks six, three, and four on three different starts. (Once the count is at least the largest the data ever need, adding more classes barely changes the overall fit, so the tuning is choosing among near-ties and can land anywhere, even at the cap of six.) There is no single count that is right, so the most standard model-selection step returns an incoherent answer.

The flexible mixture ties the best fixed-count mixture on overall fit — it is the same kind of model, only read differently — and its per-input reading recovers the count the fixed methods cannot. The held-out advantage at probe inputs of known count:

Family	Input (true count)	Held-out advantage
Rising spacing	0.0 (one)	-0.043
Rising spacing	0.5 (two)	+0.011
Rising spacing	1.0 (two)	+0.129
Staircase	0.1 (one)	-0.009
Staircase	0.5 (two)	+0.147
Staircase	0.9 (three)	+0.201
Hump	0.0 (one)	-0.018
Hump	0.5 (two)	+0.149
Hump	1.0 (one)	-0.026

Wherever the true count is one the advantage is essentially zero or slightly negative — the extra classes earn nothing. Wherever it is two or three the advantage is clearly positive, and on the staircase it grows with the count, about +0.15 at two and +0.20 at three. On the hump family it rises and falls with the truth — near zero at the one-bell-curve ends, clearly positive at the two-bell-curve middle — the case the input coordinate alone could not produce. The point predictor and the fixed-count mixture have no per-input count to report at all. On the rising-spacing family the advantage at the midpoint is only weakly positive, +0.011, because there the two bell-curves sit almost exactly at the one-peak-or-two boundary of Appendix B; it becomes clear, +0.129, once they stand well apart. These magnitudes depend on the particular fit, and run somewhat larger here — this is the flexible mixture read directly — than for the prior-route model of Section 13 (Figure 2) on the same data; the pattern, near zero at a true count of one and clearly positive above it, is what carries across.

15. Putting the prefer-few prior on trial

The flexible mixture of Section 14 was fit with the prefer-few prior of Sections 4–13. So its success leaves one question open: does the prior do the work, or does the held-out reading? The clean way to tell is to change only the prior and hold everything else fixed.

The prior enters the objective through the single term $(\alpha_0 - 1) \sum_c \ln \bar{w}_c$ of Section 13. Setting the concentration to $\alpha_0 = 1$ makes the coefficient $(\alpha_0 - 1)$ exactly zero, so the term vanishes and the model is fit by the plain

probability of the data with no prefer-few preference at all — the identical network, the prior simply switched off. We compare $\alpha_0 = 0.1$ (prior on, preferring few) against $\alpha_0 = 1$ (prior off), reading both by the same held-out question against the same single-bell-curve baseline, in the two regimes the model can sit in:

- *fixed classes*: each class is pinned to a fixed location and only the weights move with the input — the prior’s home ground, where the weights alone must carry the count, so a usage prior can bite;
- *moving classes*: each class can slide to fit the data — the regime the real model is in, since its per-class heads adapt — where the weights can stay flat while the classes themselves do the work.

Alongside, as an outside check, a plain mixture density network — a different flexible mixture with no prefer-few prior of any kind — read the same way.

Overall fit (gap to the best achievable, in nats; three families, three starts):

Family	Single bell-curve	On, fixed	Off, fixed	On, moving	Off, moving	Plain mixture
Rising spacing	+0.041	+0.027	+0.020	+0.018	+0.036	+0.022
Staircase	+0.133	+0.110	+0.065	+0.034	+0.038	+0.028
Hump	+0.043	+0.055	+0.038	+0.012	+0.033	+0.029

With the classes pinned, switching the prior off improves the fit on every family — the prefer-few pressure was costing accuracy. With the classes free to move, the differences are small and run both ways, and the plain mixture is at or near the best fit throughout. Nowhere does the prior buy a meaningful improvement in fit.

The reading that decides the matter is the held-out advantage, and the hump family shows it most sharply (true count one, two, one across the three probe inputs):

Reading	input 0.0 (one)	input 0.5 (two)	input 1.0 (one)
Prior on, fixed	-0.028	-0.035	-0.046
Prior off, fixed	+0.004	+0.043	-0.009
Prior on, moving	-0.009	+0.167	-0.009
Prior off, moving	-0.076	+0.149	-0.051
Plain mixture	-0.019	+0.138	-0.052

A faithful reading is clearly positive at the two-bell-curve middle and near zero at the ends. In the regime the real model occupies — moving classes — prior on, prior off, and the plain mixture all do this, and they are indistinguishable: the middle reads +0.167, +0.149, and +0.138, the ends near zero. The prior changes nothing where it matters. With the classes pinned, the prior actively breaks the reading: prior on reports a *negative* advantage at the two-bell-curve middle, -0.035, missing the structure entirely, because the prefer-few pressure has emptied the very class the data needed; prior off at least keeps that middle positive, +0.043. The staircase tells the same story; at its two- and three-bell-curve inputs, with the classes free to move:

Reading	input 0.5 (two)	input 0.9 (three)
Prior on, moving	+0.156	+0.202
Prior off, moving	+0.138	+0.181

Reading	input 0.5 (two)	input 0.9 (three)
Plain mixture	+0.154	+0.207

all three in step.

The engaged-class count tells the complementary half. With the classes pinned it stays modest and the prior keeps it modest; with the classes free to move it inflates well past the truth for every setting, prior or no prior, because overlapping classes cost nothing in fit. That is exactly why counting engaged classes over-counts in the regime the real model is in, and why the held-out advantage — which stays honest in both regimes — is the reading to trust.

The verdict is clean. Put on a like-for-like trial, the prefer-few prior earns nothing in the regime the real model occupies and damages the reading where the classes are pinned. The held-out advantage recovers the per-input count without it, on this model and on a plain mixture alike.

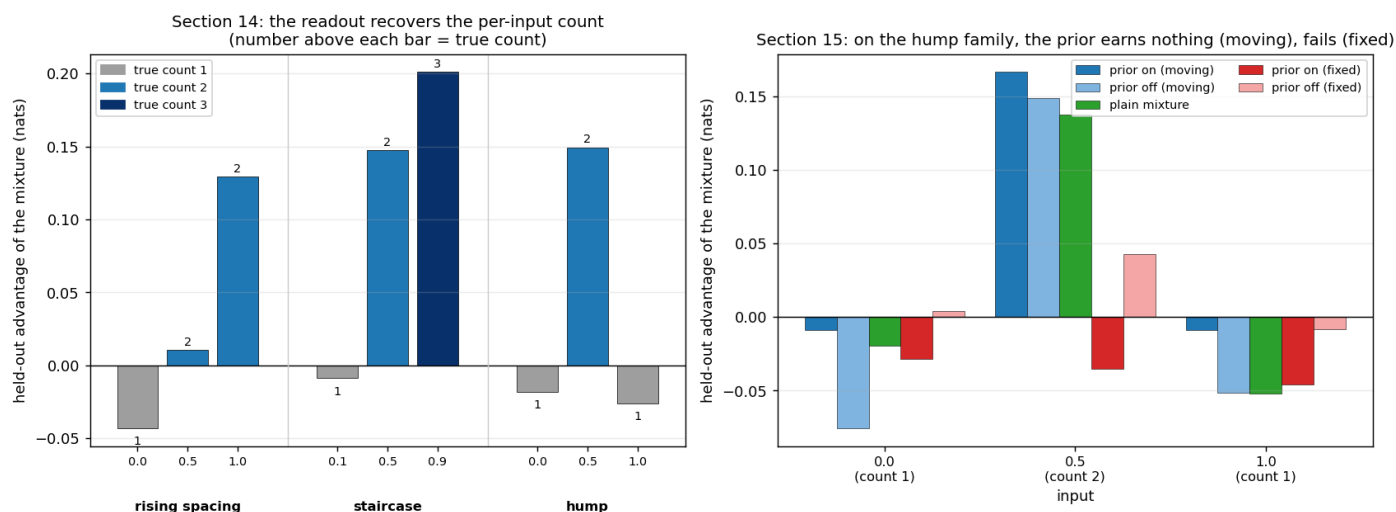


Figure 3: **The two results, side by side.** Left (Section 14): the held-out advantage of the flexible mixture over a single bell-curve at probe inputs of known true count, across the three families; each bar is coloured by the true count there and labelled with it. The advantage is near zero wherever the true count is one and clearly positive above it — on the staircase it grows from two to three, on the hump it rises and falls with the truth. Right (Section 15): the same advantage on the hump family (true count one, two, one) for the prefer-few prior on and off with the classes free to move, for the plain mixture, and for the prior on and off with the classes pinned. The three moving-class readings rise together at the two-bell-curve middle and fall at the ends; with the classes pinned the prior fails at the middle, the prior-on case going negative where the structure is. Both panels read the advantage in nats, the unit of log-probability under natural logarithms; a positive value means the extra classes predict held-out targets better than one bell-curve.

16. What this means, and what is still open

The contribution is the reading, not the prior. Across every controlled family the per-input count is recovered by one move — fit a flexible mixture, then ask on held-out data whether the extra classes predict fresh targets

better than a single bell-curve, locally around each input — and that move needs no prior, no penalty, and no count chosen in advance. It works on the classification-bottleneck model of this note and on a plain mixture model equally, so it is a property of the reading, not of any special model.

This sharpens the model’s design rather than abandoning it. The model is still a regressor regularised through a classification bottleneck; nothing about the network changes. What changes is only that the number of classes is no longer something the fit is asked to decide — not by a hand-set penalty, which never worked (Section 2), and not by a prefer-few prior, which works but is unnecessary (Sections 14–15). The prior was a sound hypothesis, developed in full and tested on its own terms; the evidence is that it does not earn its place, and the cleaner method keeps the flexible fit and the held-out reading and lets the prior go. That recommendation is set out here for the reader to weigh, not asserted as already done.

Two things remain open. First, the building blocks here are not themselves new, and the contribution has to be positioned with that in mind. Comparing a mixture against a single bell-curve by a likelihood-ratio test, to ask how many modes a body of data shows, is long established; choosing the number of mixture pieces by held-out probability is standard model selection; and letting that number depend on the input is the everyday setting of conditional mixture models. What is not standard is doing the comparison *per input* — recovering a separate count at each input from single held-out targets by averaging neighbours, checked against an exactly computable answer — and showing that this held-out reading, not the count of surviving classes, is the faithful one once the classes are free to move. Whether that combination is genuinely new against the conditional-density and mixture-testing literature is worth a proper search before building on it. Second, and more important, every result here is on small, one-dimensional, controlled data where the true count is known and “near a given input” is a short interval on a line. The real test is the same reading on real, many-dimensional targets, where there is no known answer to grade against and “near a given input” must be defined by nearest neighbours in many dimensions rather than by a position on a line. Whether the held-out advantage stays a faithful, low-noise reading there is what decides whether this is a method or only a demonstration.

Appendix A. The bound and its update rules

This appendix derives, for Basis A, the bound of Section 5 and the two update rules, starting from the quantity we want and using only standard steps.

The quantity we want. We want the weights averaged out, maximising the log-evidence $\ln p(y \mid x) = \ln \int p(w) \prod_i (\sum_c w_c \phi_{ic}) dw$. Attach a class label $z_i \in \{1, \dots, K\}$ to each point — so w_{z_i} is the weight of whichever class z_i names and ϕ_{iz_i} that class’s fit at point i (a subscript whose value is itself an index). The product of sums then expands, by the distributive law, into a sum over every way of choosing one class for each point,

$$\prod_i \sum_c w_c \phi_{ic} = \sum_z \prod_i w_{z_i} \phi_{iz_i}, \quad z = (z_1, \dots, z_N) \text{ ranging over all such choices,} \quad (13)$$

so the evidence is an average over both the weights and these labels. The integral and sum have no closed form.

The bound. Introduce any joint stand-in distribution $q(w, z)$ over the weights and the labels. Multiplying and dividing inside the logarithm by q and using that the logarithm of an average is at least the average of the

logarithm (Jensen’s inequality — true because the logarithm curves downward, so moving it inside an average can only lower the value):

$$\ln p(y | x) = \ln \mathbb{E}_q \left[\frac{p(w) \prod_i w_{z_i} \phi_{iz_i}}{q(w, z)} \right] \geq \mathbb{E}_q \left[\ln \frac{p(w) \prod_i w_{z_i} \phi_{iz_i}}{q(w, z)} \right] =: \mathcal{L}. \quad (14)$$

The gap between the evidence and $\mathcal{L}$ is exactly the Kullback–Leibler divergence from q to the true posterior of the unknowns, which is non-negative and zero only when q equals that posterior; so making $\mathcal{L}$ large both tightens the bound and drives q toward the posterior. We take a *mean-field* stand-in — the assumption that the unknowns are independent under q — of the form $q(w, z) = q(w) \prod_i q(z_i)$ with $q(w) = \text{Dirichlet}(\gamma)$ and $q(z_i = c) = r_{ic}$ — the same responsibility introduced in Section 5.

Expanding the bound. Substituting and using that z_i and w are independent under q ,

$$\mathcal{L} = \underbrace{\mathbb{E}_q[\ln p(w)] - \mathbb{E}_q[\ln q(w)]}_{- \text{KL}(q(w) \| p(w))} + \sum_i \sum_c r_{ic} (\ln \phi_{ic} + \mathbb{E}_q[\ln w_c] - \ln r_{ic}), \quad (15)$$

which is the bound quoted in Section 5. The one average needed is $\mathbb{E}_q[\ln w_c]$. For $w \sim \text{Dirichlet}(\gamma)$ the density is proportional to $\prod_c w_c^{\gamma_c - 1}$, so $\ln w_c$ is its natural sufficient statistic; for an exponential family the mean of the sufficient statistic equals the derivative of the log-normaliser, and differentiating the Dirichlet’s log-normaliser $\sum_c \ln \Gamma(\gamma_c) - \ln \Gamma(\sum_j \gamma_j)$ with respect to γ_c gives the standard identity

$$\mathbb{E}_q[\ln w_c] = \psi(\gamma_c) - \psi(\sum_j \gamma_j), \quad \psi(t) = \frac{d}{dt} \ln \Gamma(t), \quad (16)$$

with ψ the digamma function. The divergence between the two Dirichlets in the bound — the single cost that replaces the steering term — has the closed form

$$\begin{aligned} \text{KL}(\text{Dirichlet}(\gamma) \| \text{Dirichlet}(\alpha_0 \mathbf{1})) &= \ln \Gamma(\sum_c \gamma_c) - \sum_c \ln \Gamma(\gamma_c) - \ln \Gamma(K\alpha_0) + K \ln \Gamma(\alpha_0) \\ &\quad + \sum_c (\gamma_c - \alpha_0) (\psi(\gamma_c) - \psi(\sum_j \gamma_j)), \end{aligned} \quad (17)$$

obtained as follows: the log-density of a Dirichlet is $\ln \text{Dirichlet}(w; \beta) = \ln \Gamma(\sum_c \beta_c) - \sum_c \ln \Gamma(\beta_c) + \sum_c (\beta_c - 1) \ln w_c$; taking $\mathbb{E}_q[\ln q - \ln p]$ with $q = \text{Dirichlet}(\gamma)$ and $p = \text{Dirichlet}(\alpha_0 \mathbf{1})$, the $\ln \Gamma$ terms come out directly and each $\ln w_c$ averages to $\psi(\gamma_c) - \psi(\sum_j \gamma_j)$ by the identity above, leaving the terms shown. It depends only on γ , not on the responsibilities, so it sits out the responsibility update and contributes only to the weight update, which is derived independently below.

The responsibility update. Hold γ fixed and maximise $\mathcal{L}$ over one point’s responsibilities r_i , which must sum to one. Only the double sum involves r_i ; with a Lagrange multiplier λ enforcing the sum-to-one constraint (the standard way to optimise subject to a constraint; differentiating $-r_{ic} \ln r_{ic}$ gives $-\ln r_{ic} - 1$, which supplies both the $-\ln r_{ic}$ and the -1 below), setting the derivative to zero,

$$\frac{\partial}{\partial r_{ic}} \left[r_{ic} (\ln \phi_{ic} + \mathbb{E}_q[\ln w_c] - \ln r_{ic}) + \lambda r_{ic} \right] = \ln \phi_{ic} + \mathbb{E}_q[\ln w_c] - \ln r_{ic} - 1 + \lambda = 0, \quad (18)$$

so $r_{ic} \propto \phi_{ic} \exp(\mathbb{E}_q[\ln w_c]) = \phi_{ic} \exp(\psi(\gamma_c)) / \exp(\psi(\sum_j \gamma_j))$. The common factor cancels on normalising over c , leaving the update of Section 5, $r_{ic} = \phi_{ic} \exp(\psi(\gamma_c)) / \sum_j \phi_{ij} \exp(\psi(\gamma_j))$.

The weight update. Hold the responsibilities fixed and maximise over $q(w)$. The mean-field optimum for one factor sets its log equal to the average of the log joint over the other factors, plus a constant. The only weight-dependent terms are the prior and the labels' contribution:

$$\ln q^*(w) = (\alpha_0 - 1) \sum_c \ln w_c + \sum_i \sum_c r_{ic} \ln w_c + \text{const} = \sum_c ((\alpha_0 + N_c) - 1) \ln w_c + \text{const}, \quad (19)$$

with $N_c = \sum_i r_{ic}$ (each “const” above collects terms that do not involve w). This is the log of a Dirichlet with concentrations $\gamma_c = \alpha_0 + N_c$ — the update of Section 5. The two updates are applied in alternation; the network producing $\mu_c(x)$ and $\sigma_c(x)$ takes gradient steps on the same bound in between, which for fixed weights is just the responsibility-weighted sum of the class log-fits, $\sum_i \sum_c r_{ic} \ln \phi_{ic}$ — the same quantity the responsibility step already uses.

Appendix B. When two bell-curves make one peak or two

Two equal-weight bell-curves of equal width σ with centres at $\pm a$ — so a distance $d = 2a$ apart — give the density, up to a constant,

$$f(y) \propto \exp\left(-\frac{(y-a)^2}{2\sigma^2}\right) + \exp\left(-\frac{(y+a)^2}{2\sigma^2}\right). \quad (20)$$

By symmetry $f'(0) = 0$, so the midpoint is always a turning point; whether it is a single central peak or the dip between two peaks is decided by the curvature there. Differentiating twice and evaluating at $y = 0$,

$$f''(0) = \frac{2}{\sigma^2} \exp\left(-\frac{a^2}{2\sigma^2}\right) \left(\frac{a^2}{\sigma^2} - 1\right). \quad (21)$$

The midpoint is a minimum — a dip between two separated peaks — exactly when $f''(0) > 0$, that is when $a > \sigma$, meaning the centres are more than 2σ apart. Below that the two overlap into a single peak. This is the boundary $s = d/\sigma = 2$ used in Section 11 to mark where the conditional crosses from one effective mode to two.

References

- Corduneanu, Adrian, and Christopher M. Bishop. 2001. “Variational Bayesian Model Selection for Mixture Distributions.” In *Proceedings of the 8th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 27–34.
- Rousseau, Judith, and Kerrie Mengersen. 2011. “Asymptotic Behaviour of the Posterior Distribution in Overfitted Mixture Models.” *Journal of the Royal Statistical Society: Series B (Statistical Methodology)* 73 (5): 689–710. <https://doi.org/10.1111/j.1467-9868.2011.00781.x>.
- Teicher, Henry. 1963. “Identifiability of Finite Mixtures.” *The Annals of Mathematical Statistics* 34 (4): 1265–69. <https://doi.org/10.1214/aoms/1177703862>.